2025/11/12



IntelliJ IDEA plugin

TA-Assistant

汇报人：2350280 程瑜 2351882 王小萌 2351759 程琮越 2352996 路清怡

CONTENTS

目录 CONTENTS

01 System Overview

02 Overall Architecture

03 RAG Data Storage and Algorithms

04 LLM Integration and Prompt Engineering

05 Demonstration

06 Team Collaboration Mechanism

07 Key Problems & Solutions

08 Summary

01

System Overview

1 F0 Function

F0 function focuses on **RAG course Q&A**, supports strict source citation, has a **tool window UI** and **right-click context awareness** capability, and can accurately answer course-related questions.

2 F2 Function

F2 function focuses on AI-driven code modification, provides **Diff review** and **approval functions**, supports **streaming generation**, **editable suggestions** and **secure application** to help code optimization.

3 Supported Document Types

The system supports three common document formats: **PDF**, **Word** and **PPT**. It also has **model switching** and **state management** functions to meet diverse needs.

4 TopK and Citation Norms

The system uses the **TopK algorithm** to optimize retrieval results and strictly follows **citation norms** to ensure the accuracy and reliability of answers.

02

Overall Architecture

Four-Layer Architecture Responsibility Division

Presentation Layer

responsible for user interaction, including the Swing tool window, right-click actions, and the Diff Viewer

Data and Indexing Layer

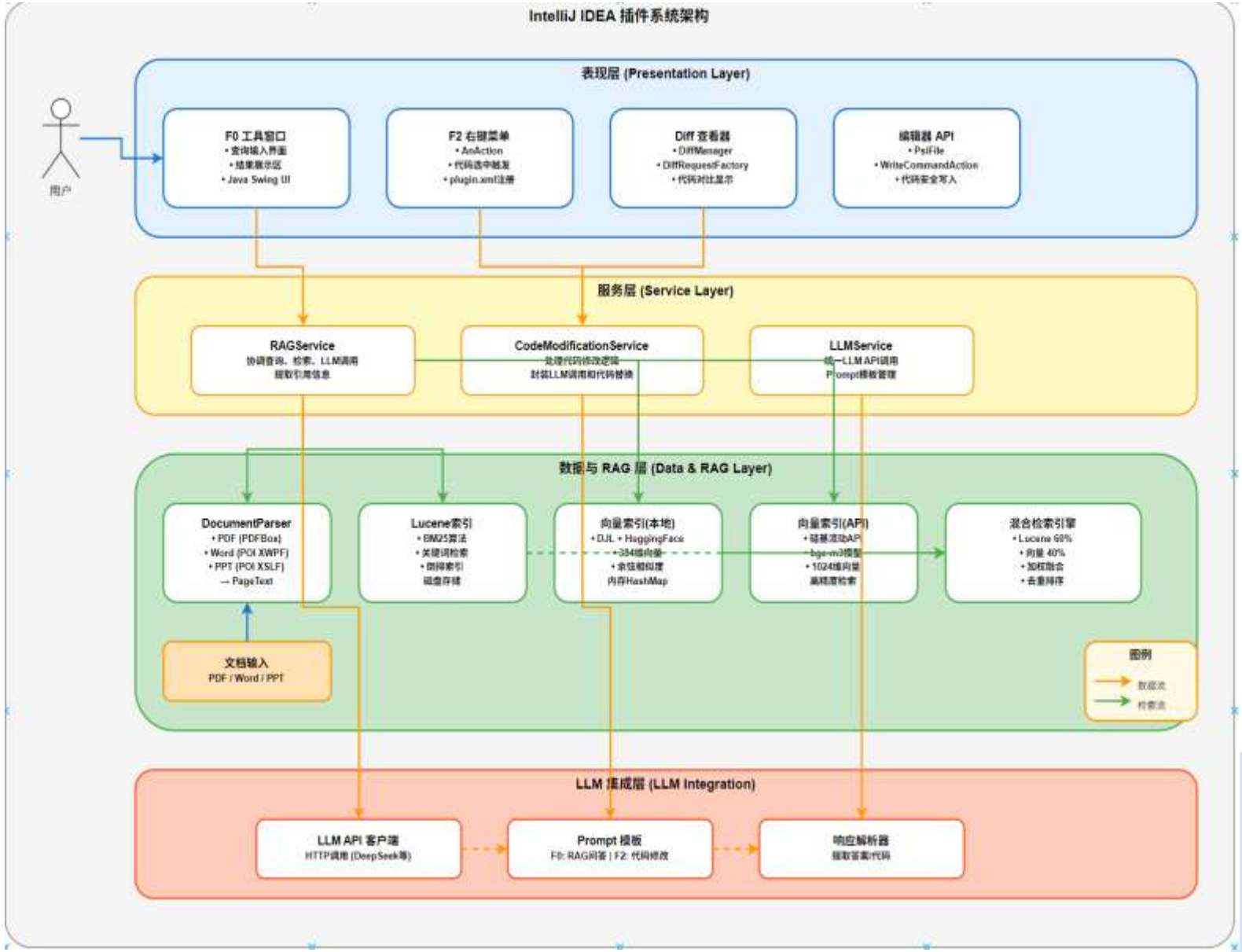
includes DocumentParser, KnowledgeStore, Chunk, and Citation.It handles data processing and indexing functions.

Service Layer

encapsulates RAGService, CodeModificationService, and LLMService,providing support for core business logic.

Integration Layer

The layer connects the unified LLM interface with response parsing, achieving coordination among different modules and ensuring smooth system operation.



03

RAG Data Storage and Algorithms

Multi-format Parsing and Chunking Strategy



WORD Parsing — Sliding Window Chunking

Chunk size: 800 characters, with 100-character overlap

Smart boundary cutting: Prefer cutting at punctuation marks in this order:

Period (。..) → Question/Exclamation marks (!?? ！ ？) → Semicolon (； ;) → Line break (\n\n) → Space

Advantage: Keeps chunk sizes balanced and ensures semantic continuity.

PDF Parsing

Each page is treated as an independent document chunk.

Preserves page metadata for precise citation.

Suitable for structured lecture notes and course materials

PPT Parsing

Each slide is treated as an independent document chunk.

Extracts titles and main text content.

Facilitates quick indexing and knowledge retrieval.

Index Structure



Lucene Inverted Index

Each document chunk’s full text is tokenized to build an inverted index for the **content** field.At the same time, the file name (**source** field) and page range **(page** field) are precisely stored as non-tokenized metadata.

The entire index is persisted locally on disk

Vector Index

Using fileName:pageNumber as the unique key (e.g., "Lecture3.pdf:5"),

each document chunk is converted into a 384- or 1024-dimensional embedding vector (float[]).

These vectors are stored in memory, with a central **HashMap** enabling constant-time **O(1)** lookup efficiency.

Search Algorithms

Lucene Keyword Search

Use StandardAnalyzer to tokenize queries.

Build multi-term BooleanQuery.

Retrieve chunks matching query terms from the inverted index.

Rank by BM25 and return Top-K relevant chunks.

Vector Similarity Search

Convert query text to a 384-dim vector: qVec = predictor.predict(query) Compare qVec with all document embeddings in HashMap

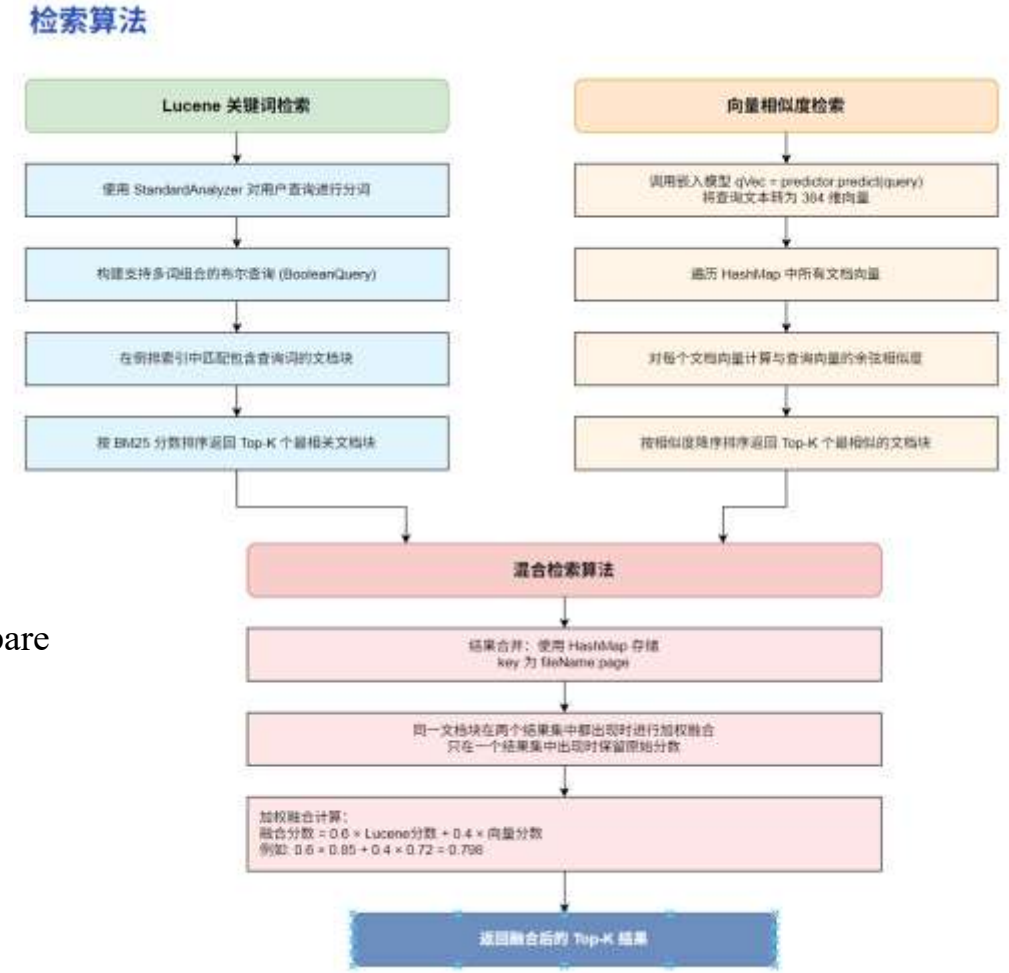
Rank by cosine similarity and return Top-K closest chunks.

Hybrid Search

Merge Lucene and vector results by key fileName:page

If a chunk appears in both, apply weighted fusion:

Example:Score = 0.6×0.85 + 0.4×0.72 = 0.798



Storage Structure

01 Search Results Encapsulated as RetrievedChunk Class

```
public class RetrievedChunk {
    private final String sourceFile;    // 源文件名（如 "lecture3.pdf", "notes.docx"）
    private final String pageRange;     // 页码/块号（如 "p5", "3"）
    private final String content;       // 文档块的完整文本内容
    private final double score;         // 相关性分数 [0, 1]，越大越相关

    public String getSourceFile() { return sourceFile; }
    public String getPageRange() { return pageRange; }
    public String getContent() { return content; }
    public double getScore() { return score; }
}
```

02 Field Descriptions

sourceFile: Displayed in the UI as the document source (e.g., "This answer is from Lecture3.pdf").

pageRange: For PDFs, represents the page number (e.g., "p5" = page 5).For Word documents, represents chunk number (e.g., "3" = the 3rd chunk).For PPT files, represents slide number (e.g., "p2" = slide 2).

content: The full text of the document chunk, used as context input to the LLM or shown as the original summary in the UI.

score: Used for ranking and confidence display.0.8–1.0: highly relevant0.6–0.8: moderately relevant0.4–0.6: weakly relevant<0.4: mostly irrelevant

04

LLM Integration and Prompt Engineering

Core Design 1： LLMClient vs SiliconLLMClient

Program to an Interface

Achieve unified access to different LLM models through the LLMService and LLMClient abstraction layers, reducing development complexity.

```
public interface LLMClient {  6 个用法  2 个实现
    /**
     * 向大语言模型发送一个提示（prompt），并获取生成的文本回复。
     * @param prompt 构造好的完整提示词
     * @return LLM 生成的回复字符串
     */
    String generate(String prompt);  3 个用法  2 个实现
}
```

SiliconLLMClient

·Manages API Keys, constructs HTTP requests, sets Headers, serializes JSON, and deserializes JSON responses.

·Interaction: Implements the LLMClient interface and interacts with the external api.siliconflow.cn HTTP endpoint within the generate method.

```
/**
 * An LLMClient implementation for the SiliconFlow API.
 */
public class SiliconLLMClient implements LLMClient {  14 个用法
```

Core Design 2： TAService and AIModificationUtil

Service Layer

·Management: currentModel

·Encapsulate F0/F2 Business Logic: TAService is responsible for RAG Q&A; AIModificationUtil modifyCode (code modification). The UI layer should not be aware of complex steps like RAG's retrieval, sorting, or Prompt construction.

·Lifecycle Management: Specifically, TAService is a singleton service initialized at program startup. It is responsible for loading and indexing the initial knowledge base during its construction.

Interaction

·Held by the UI layer.

·When answer is called, a SiliconLLMClient instance is created on-demand within the method, and the current state it manages (currentModel) is passed to it.

F0 RAG Prompt Design

```
"You are a senior Java architect and refactoring expert.\n" +
"Your task is to modify the given Java code snippet according to the user's instructions.\n" +
"Rules:\n" +
"1. Strictly follow the user's instructions for modification.\n" +
"2. Do not add any explanations, comments, or apologies unrelated to the code. " +
"Your response must be purely, directly compilable Java code.\n" +
"3. Maintain the indentation and formatting style of the original code.\n" +
"4. If the instruction cannot be completed, return the original code.\n\n" +
"\"[User Instruction]\"\n" +
instruction + "\n\n" +
"\"[Original Code]\"\n" +
"```java\n" + code + "\n" + "```\n\n" +
"\"[Modified Code]\"\n";
```

Role

“You are a teaching assistant..."

Limit the AI's role, making its answers more focused on "teaching assistance."

Enforce [GENERAL_KNOWLEDGE] Tag

When the knowledge base has no relevant content, the AI must add this tag -> Backend sets the isFallback flag -> UI displays a "General Knowledge" disclaimer.

Enforce ``` Format Purpose

To achieve machine-readable citations. Flow: Backend parses using regular expressions (CITATION_PATTERN) -> UI renders as clickable cards.

[Background Knowledge], [Code Context]

Use clear tags to delineate the knowledge area, code area, and question area to prevent the LLM from confusing different contexts.

F2 Prompt Design

[Role]

Switch to a code refactoring expert, guiding the AI to generate high-quality, compliant code.

"...must be purely, directly compilable Java code."

Force the AI to only output pure code.

"Maintain the indentation..."

Prevent the AI from breaking the user's original indentation during refactoring.

[Original Code] / ```java

Structured input. Use tags to clearly distinguish the "instruction area" from the "code area," helping the AI with precise positioning.

```
"You are a senior Java architect and refactoring expert.\n" +
"Your task is to modify the given Java code " +
" according to the user's instructions.\n" +
"Rules:\n" +
"1. Strictly follow the user's instructions for modification.\n" +
"2. Do not add any explanations, comments, or apologies unrelated to the code.
"Your response must be purely, directly compilable Java code.\n" +
"3. Maintain the indentation and formatting style of the original code.\n" +
"4. If the instruction cannot be completed, return the original code.\n\n" +
"\"[User Instruction]\"\n" +
instruction + "\n\n" +
"\"[Original Code]\"\n" +
"```java\n" + code + "\n" + "```\n\n" +
"\"[Modified Code]\"\n";
```

Response Parsing & Security Compliance

Citation Parsing

Citation Parsing Parse the citation tags returned by the model to generate standardized Citation cards, making it convenient for users to view the citation sources.

Security Check

Perform Markdown cleaning and syntax compliance checks on the code output to ensure the security and reliability of the generated code.



05

Demonstration

06

Team Collaboration Mechanism

Horizontal Division of Labor and Interface Freezing

Distinct Roles and Responsibilities

Our team conducts **parallel development along four lines**—Parsing & Retrieval, RAG Orchestration, UI Tool Window, Right-click Actions, and Project Skeleton—to enhance efficiency.

Interface Freezing

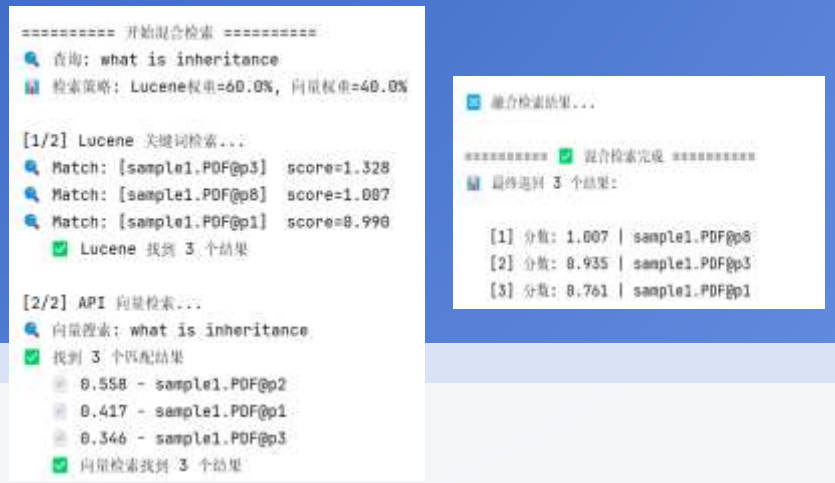
We first **freeze the three core interfaces**: KnowledgeStore, RagService, and Answer, then conduct joint debugging with mock data to **reduce blockages**.



Engineering Discipline and Configuration Management

01 Logging Standards

All **LLM call records** include response time, token count, and error types, facilitating issue tracking and performance optimization.



02 Configuration Management

API Key, index directory, and cache path are **centralized** in a unified configuration file, supporting **automatic loading** during runtime both inside and outside the IDE.

07

Key Problems & Solutions

Guarantee of Citation Accuracy

01 Page Number Preservation

DocumentParser not only records source filenames and page numbers for each text chunk , but also strictly binds text content to its original page metadata via the **List<PageText>** structure.

02 Deduplication via Unique Key

ChunkId-based deduplication and **UI-layer secondary sorting** ensure merged display of same-page fragments.

```
// 去重逻辑
Set<String> seen = new HashSet<>();
List<Citation> uniqueCitations = new ArrayList<>();
for (Citation citation : citations) {
    String key = citation.getSourceFile() + "|" + citation.getPageRange();
    if (seen.add(key)) {
        uniqueCitations.add(citation);
    }
}
if (uniqueCitations.isEmpty()) return;
```



UI Freezing and Stability Governance

Background Execution

Use **SwingWorker** to **offload time-consuming operations to the background**. The foreground only displays a progress bar and a cancel button to avoid UI freezing.

Example : AI-Powered Code Modification

1) Progress indicator: Show task title “Generating code with [Model Name]...”

2) Streaming updates: Real-time update AI-generated code blocks to the UI via SwingUtilities.invokeLater()

3) Cancel button automatically provided by Task.Backgroundable

Error Boundary

Capture unknown exceptions, prompt users with **pop-up notifications** and log errors.



Diff Review Experience Optimization



1 Streaming Output

The streaming output **cleans Markdown in real-time**, and the Diff Viewer uses a **highlighting engine** to enhance the review experience.

2 Secure Writing

Suggestions are **editable**. Back up the original file before application; roll back if canceled to ensure code security.



08

Summary



Technical Completion

Both F0 and F2 functions meet the requirements of the course assignment. Verified through unit tests and demonstration scripts, the technical completion aligns **100%** with the assignment requirements.

2025/11/12

THANKS
For Listening